XCX

CVE-2025-55182

- 先配新版环境
- 下载源码
- 部署完测试
- 插件探索
- WAF部署模板
- WAF绕过

小程序这边

小程序后端管理员界面

参考

CVE-2025-55182

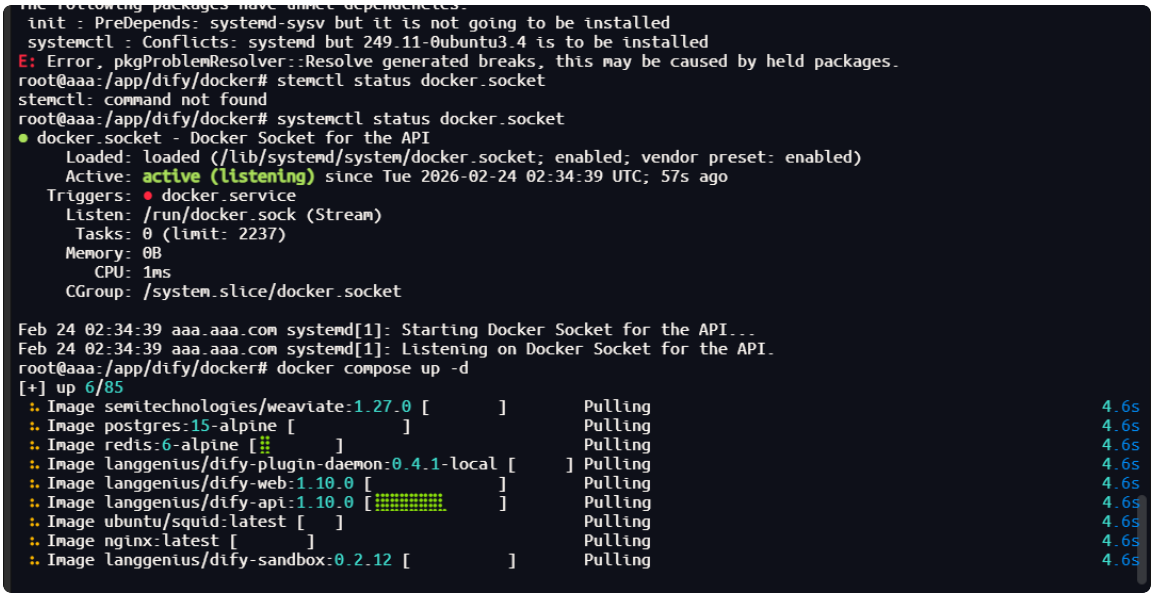
先配新版环境

```
# 1. 卸载旧版本（如有）
sudo apt-get remove docker docker-engine docker.io containerd runc

# 2. 设置仓库
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/trusted.gpg.d
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/trusted.gpg.d/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/trusted.gpg.d/docker.gpg] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 3. 安装新版 Docker
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# 4. 验证版本（≥20.10 即可）
docker --version

# 5. 此时就能用新语法了
docker compose up -d


```

```
# 停止 docker.socket 服务
sudo systemctl stop docker.socket

# 重启 docker.socket 并设置开机自启
sudo systemctl enable --now docker.socket

# 检查 docker.socket 状态（需显示 active (running)）
sudo systemctl status docker.socket
```

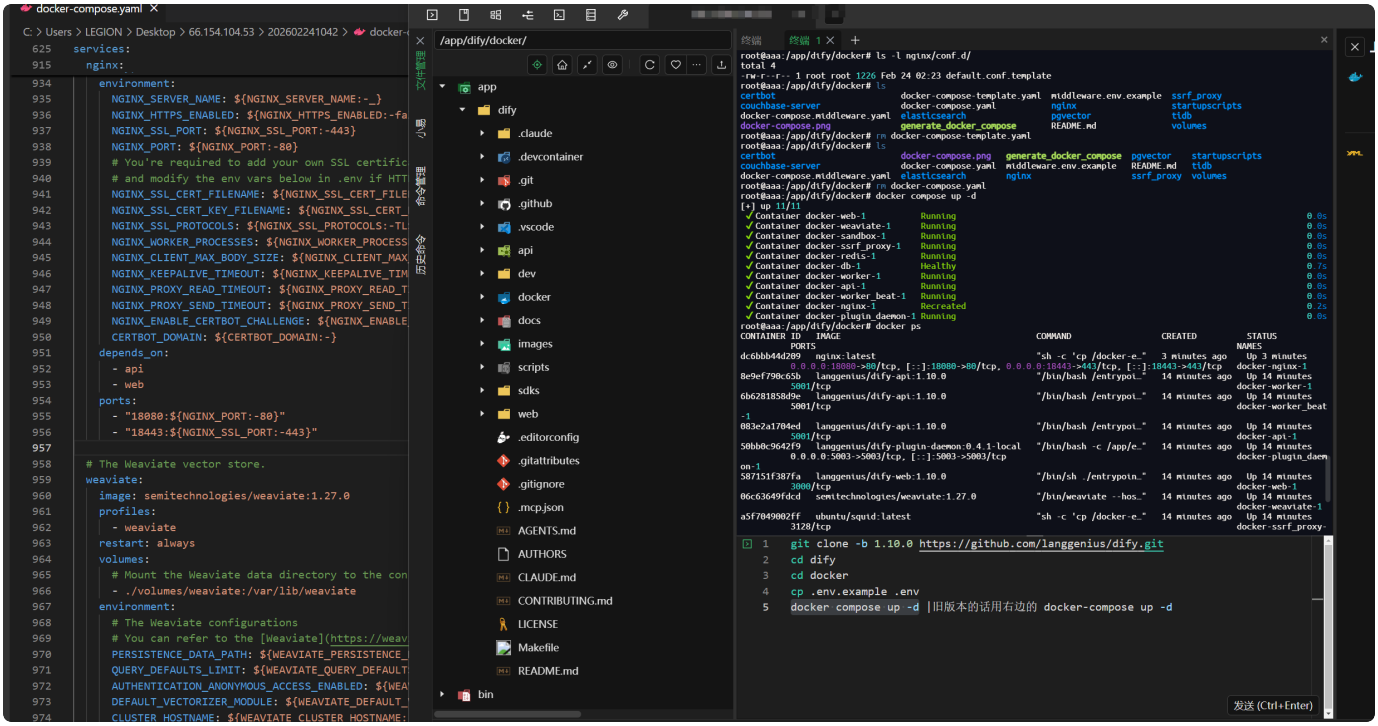
下载源码

```plain text
git clone -b 1.10.0 https://github.com/langgenius/dify.git
cd dify
cd docker
cp .env.example .env
docker compose up -d |旧版本的话用右边的 docker-compose up -d

docker compose stop
```



部署完测试

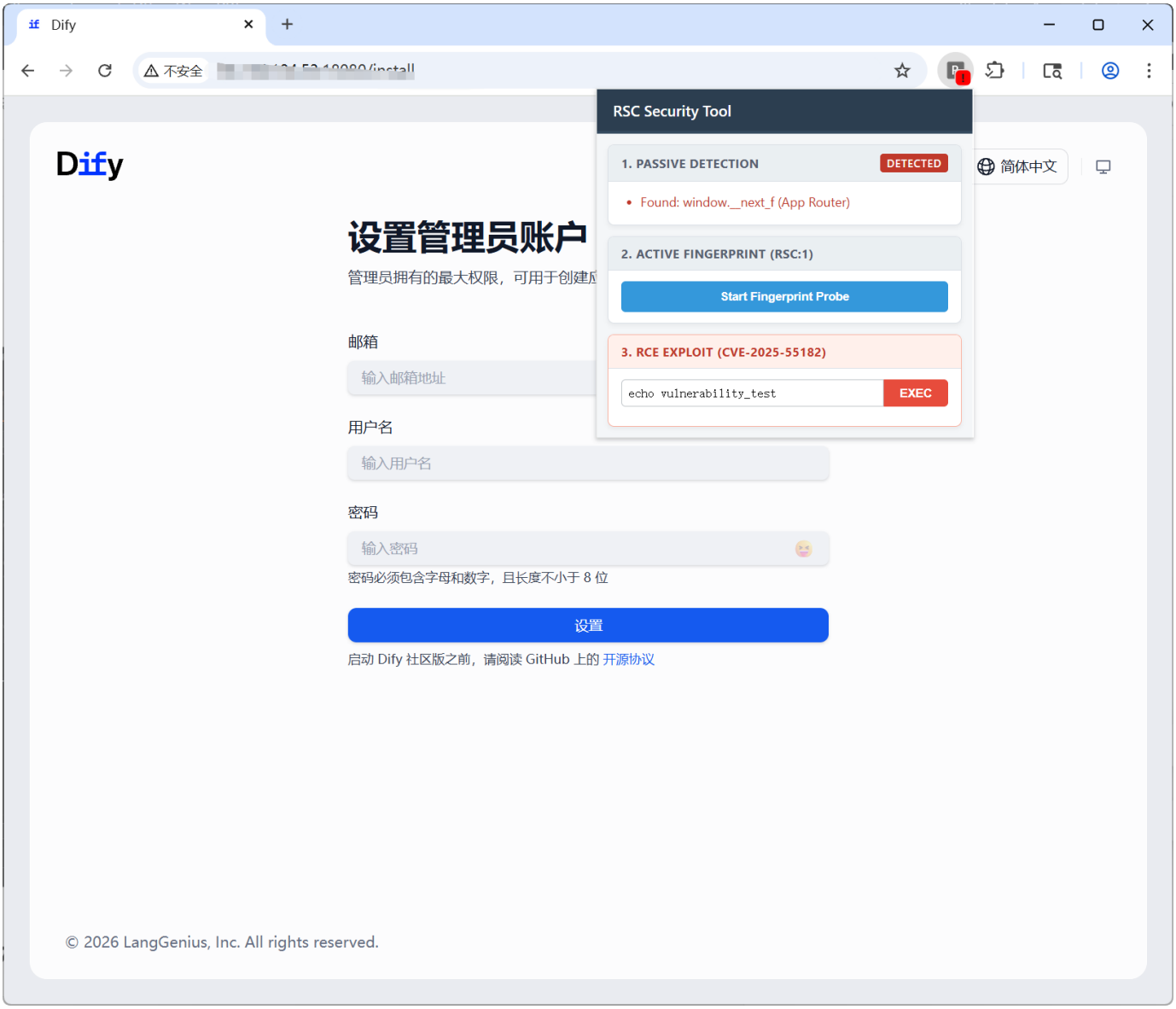
我这个端口换到了18080和18443了，因为端口不够用了



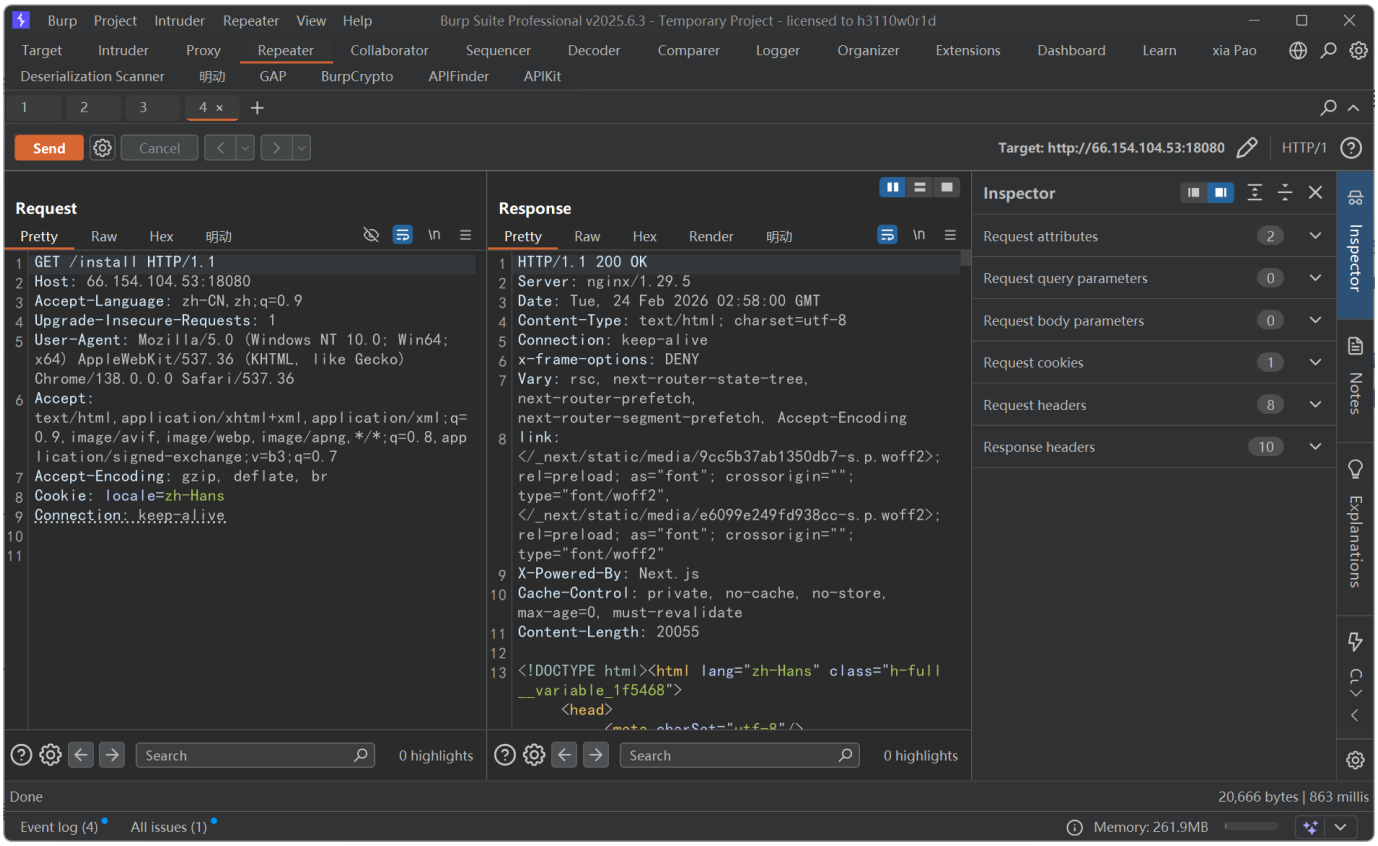
Plain Text

```
1
```

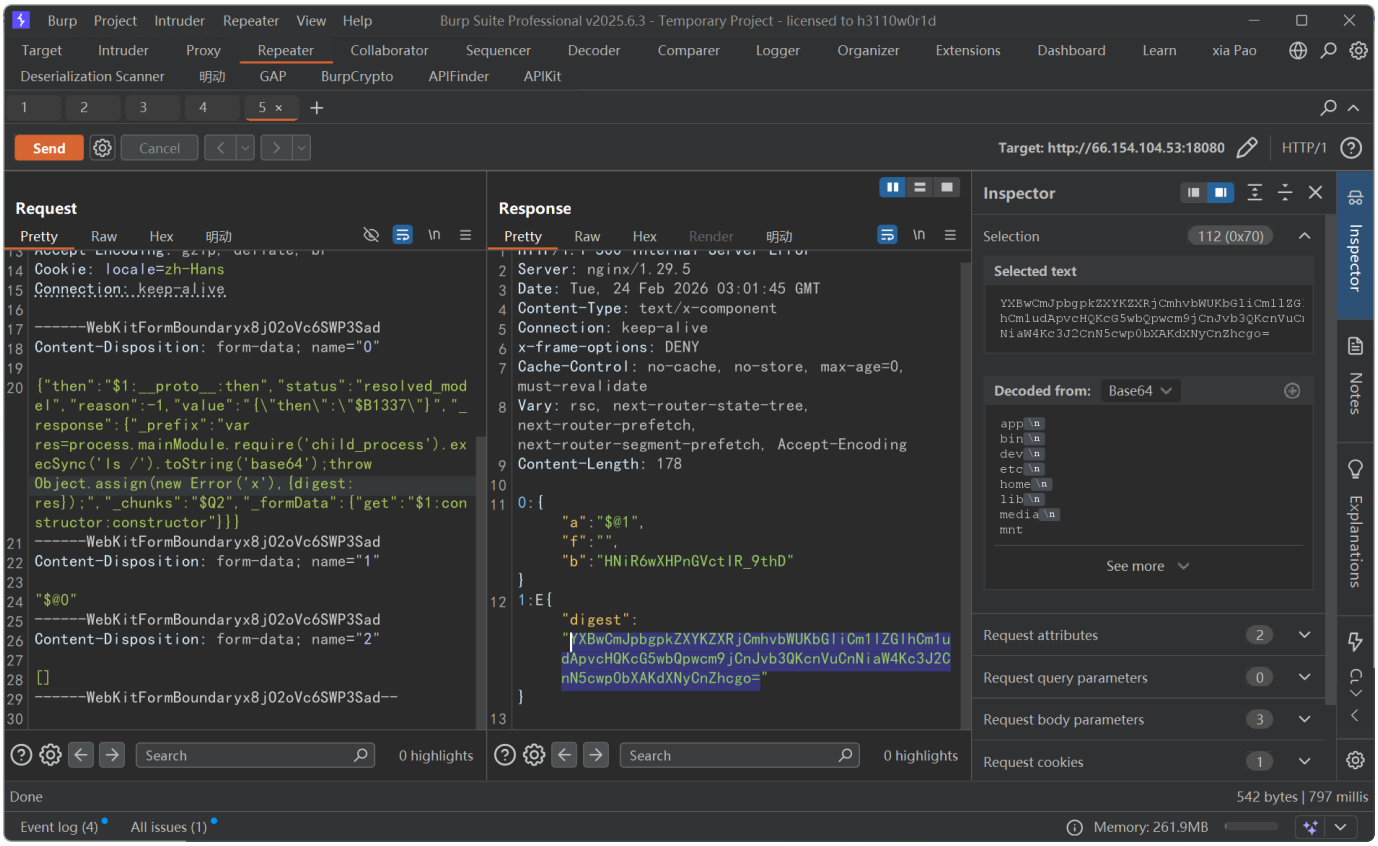
插件探索



研究研究检测逻辑，第一个包他就能识别了



命令执行的发包



WAF部署模板

Plain Text

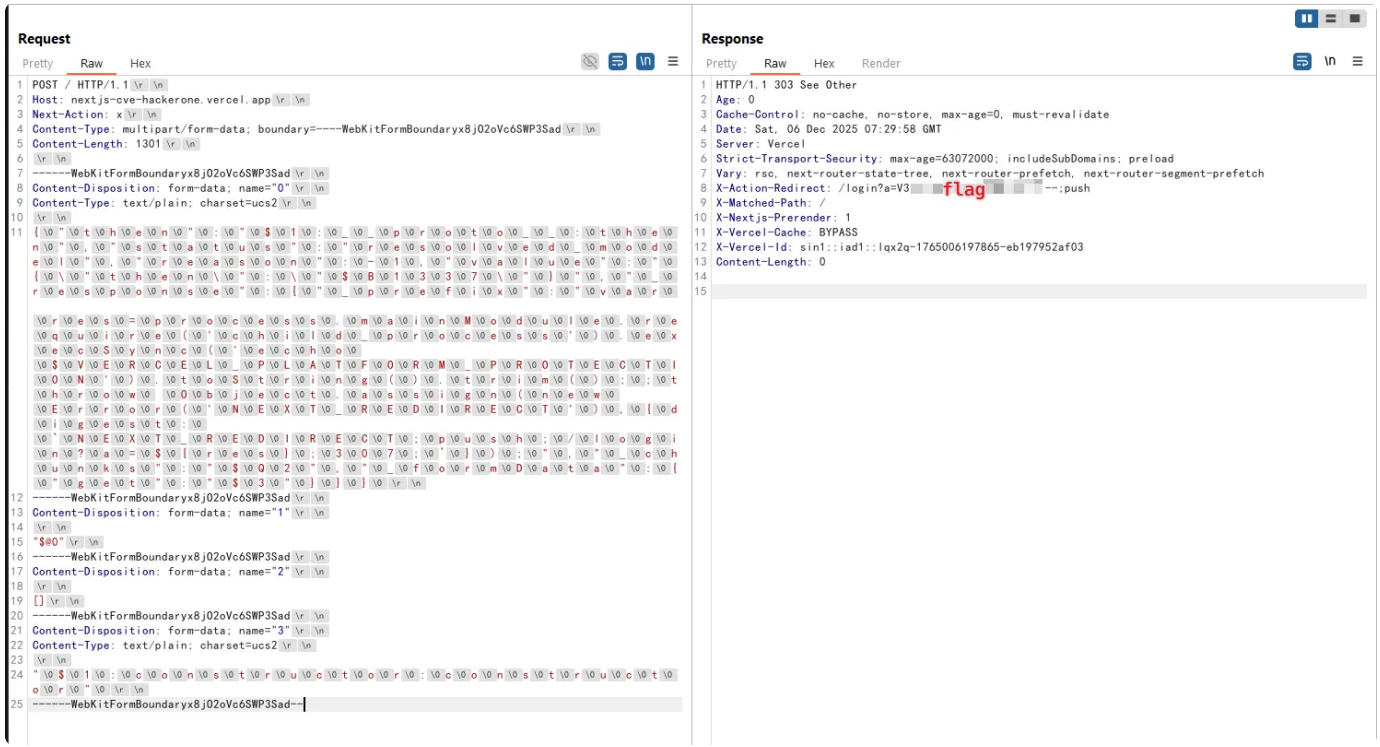
```
# www.lidd8.cn 的 Nginx 站点配置
server {
    listen 80;
    listen 443 ssl http2;
    server_name www.lidd8.cn;

    # ===== 新增：SSL 基础配置（如果需要 HTTPS）=====
    ssl_certificate /path/to/your/cert.pem;
    ssl_certificate_key /path/to/your/key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;

    # ===== 核心：CVE-2025-55182 防护规则 =====
    # 1. 开启请求体读取（必须，否则无法检测 POST 内容）
    client_body_buffer_size 1M;
    client_max_body_size 10M;

    # 2. 定义攻击特征匹配规则
    map $request_body $block_rce {
        default 0;
        ~*(?i)next-action|rsc-action-id 1;  # RSC 核心特征
        ~*(?i)#__proto__|#constructor 1;    # 原型污染特征
        ~*(?i)child_process|fs#write|exec 1; # 命令执行特征
    }

    # 3. 拦截逻辑：仅对 POST 请求生效，匹配特征则返回 403
    if ($request_method = POST) {
        if ($block_rce = 1) {
            return 403;
        }
    }

    # ===== 原有站点配置（请替换为你的实际内容）=====
    root /var/www/www.lidd8.cn;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ /index.html;
        # 如果是 Next.js 应用，需添加：
        # proxy_pass http://127.0.0.1:3000;
        # proxy_set_header Host $host;
        # proxy_set_header X-Real-IP $remote_addr;
    }

    # 日志配置
```

```
        access_log /var/log/nginx/www.lidd8.cn.access.log;
        error_log /var/log/nginx/www.lidd8.cn.error.log;
    }
```

WAF绕过

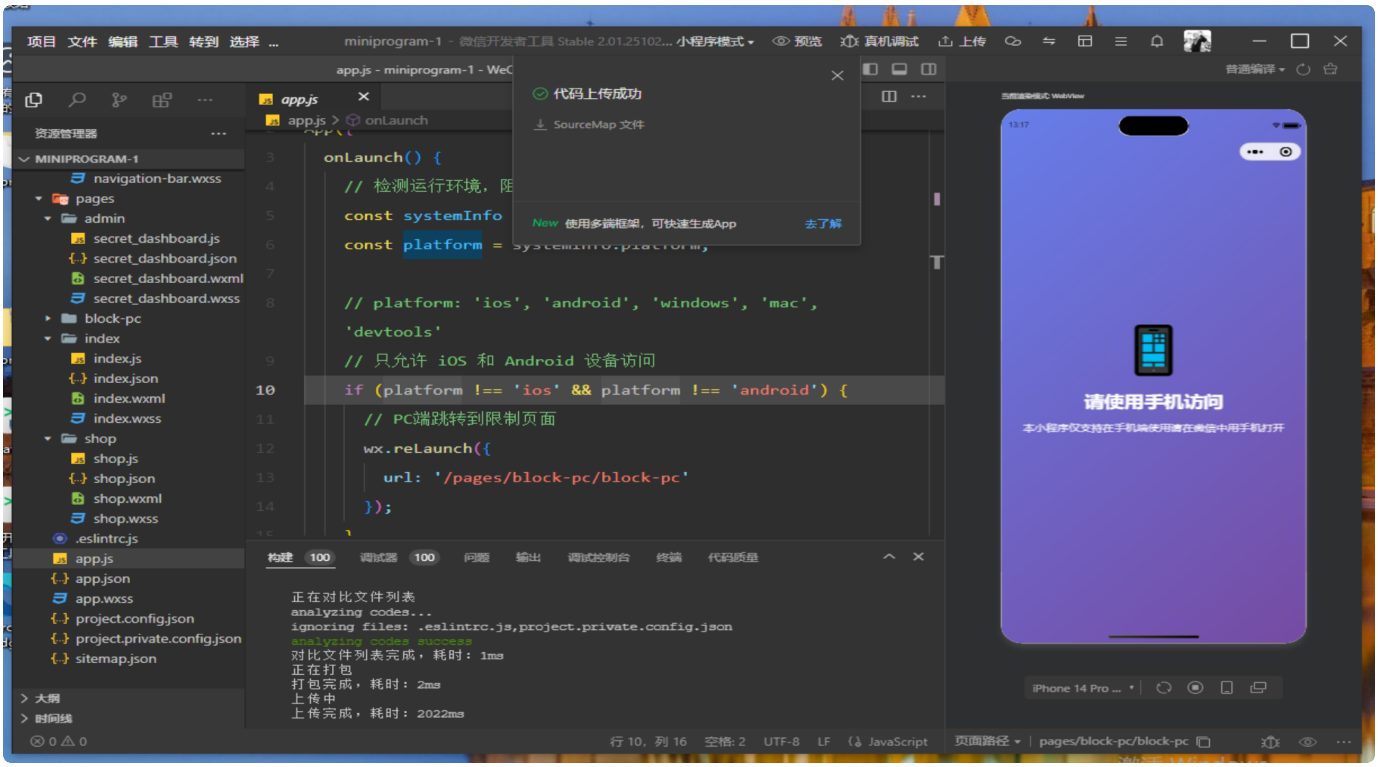
Plain Text

```
p牛绕过waf，利用编码

next.js在解析multipart包的时候使用了busboy这个库，这个库实现了RFC 7578中声明的charset选项，所以我们可以通过一些特殊的charset绕过WAF对CVE-2025-55182的拦截，比如利用utf16le编码。
```



```
(venv-55182) root@aaa:/app/CVE-2025-55182-Waf# python3 CVE-2025-55182.py -u http://66.154.104.53:18080/install

Author: l0n3m4n  | CVE-2025-55182 | Next.js/React RSC Scanner & Exploit

[+] VULNERABLE: http://66.154.104.53:18080/install - Status: 500 (Mode: rce) (OS: Linux, Web Server: nginx/1.29.5)
```

小程序这边



Plain Text

1

小程序后端管理员界面

```
# 选择官方Tomcat镜像作为基础镜像，这里以Tomcat 9为例（也可换为8.5、10等）
FROM tomcat:9.0-jdk8

# 维护者信息（可选，仅作标识）
MAINTAINER ctf-user

# 将本地的war包复制到Tomcat的webapps目录（Tomcat启动时会自动解压war包）
# 注意：COPY 本地war包名 容器内路径
COPY test.war /usr/local/tomcat/webapps/

# 暴露Tomcat默认端口8080（可选，仅作声明，运行时仍需-p映射）
EXPOSE 8080

# 启动Tomcat（官方Tomcat镜像已默认设置此命令，可省略）
CMD ["catalina.sh", "run"]
```

```
docker build -t ctf-tomcat:v1 .

docker run -d -p 52111:8080 --name ctf-tomcat-container ctf-tomcat:v1

docker rm ctf-tomcat-container
```

参考

【dify配置】

https://cloud.tencent.com/developer/article/2607793